

2110531 - Data Science and Data Engineering Tools

Spark ML

Asst.Prof. Natawut Nupairoj, Ph.D.
Department of Computer Engineering
Chulalongkorn University
natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Outline

1

The ML problem

- Predict driver arrival time
- Label, features, leakage
- Time split and baseline

2

Spark ML concepts

- DataFrames and vectors
- Transformers and Estimators
- Params and feature stages
- Pipelines

3

The tutorial, step by step

- Build the training table
- One helper, many pipelines
- Results and error analysis

4

Beyond the tutorial

- Other algorithms
- Tuning
- Saving and loading models
- Spark ML at scale

Notebook: case/ride-hailing/ml/tutorial_spark_ml.ipynb · data: synthetic MetroRide trips

Part 1

The ML problem



When will my driver arrive?

When a driver accepts a trip, predict how many minutes until they reach the pickup point.

“How long will my driver take to reach me?”

Passenger, right after the driver accepts

- Label: minutes from request to arrival
- Features: only values known at request accepted
- Data: synthetic trips, June–September; rainfall for September only

501

labelled trips (completed, with an arrival event)

8.0

minutes average wait

2.4

minutes standard deviation

4

service zones: Siam, Asok, Ari, Silom

CHULA ENGINEERING
Foundation toward Innovation

Recap: an ML problem is a table

- One row = one example
  - a completed MetroRide trip
- Label: the value we predict
  - driver_arrival_wait_minutes
- Features: inputs known at the prediction moment
  - zone, acceptance delay, rain
- Regression: the label is a number

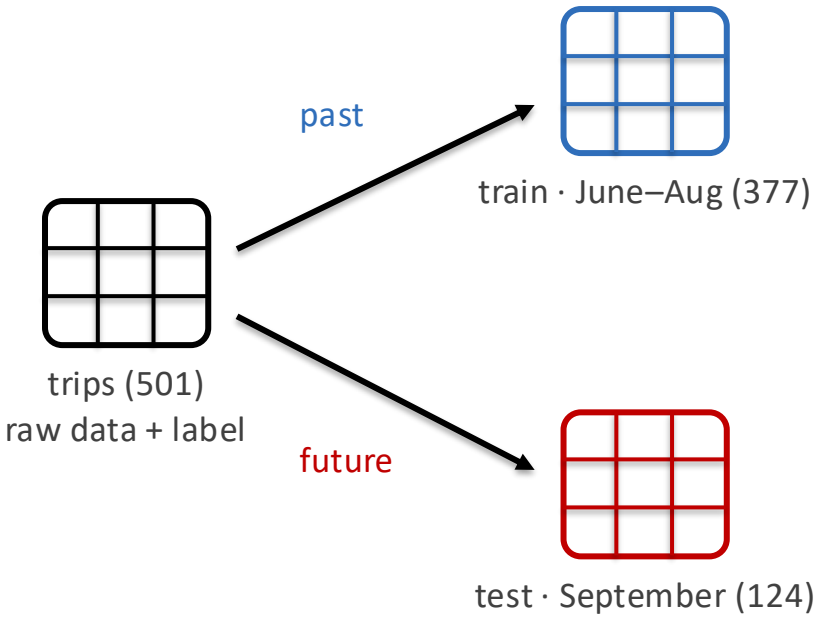
service_zone	accept_delay	is_rainy	wait (label)
Ari	2.0	0.0	6.0
Ari	1.0	1.0	11.0
Silom	2.0	1.0	13.0
Siam	3.0	1.0	10.0

features: known when the driver accepts

label: later

Leakage: a feature that is not known yet at the prediction moment. Passenger pickup time happens after the driver arrives, so it can never be a feature.

Recap: split by time, beat a baseline




```
train = data.filter(F.col("month") <= 8)
test  = data.filter(F.col("month") == 9)
```

Split by time, not at random

In production we predict the future from the past, so we test that way.

Always score a baseline

Predict the training mean for every trip: MAE 1.97 minutes. A model must beat it.

Metrics for regression

MAE: average |label − prediction| · RMSE: big misses count more · R^2: share of variance explained

Part 2

Spark ML concepts



https://pragmaedge.com/beginners-guide-to-machine-learning-with-apache-spark/

What is in Spark ML?

Algorithms

Classification, regression, clustering, recommendation

`LinearRegression`, `LogisticRegression`, `RandomForestRegressor`, `KMeans`, `ALS`

Feature transformers

Index, encode, scale and assemble features

`StringIndexer`, **`OneHotEncoder`**, **`VectorAssembler`**, `StandardScaler`

Pipelines

Chain stages into one Estimator

`Pipeline`, **`PipelineModel`**

Evaluation

Turn predictions into one metric

`RegressionEvaluator`, `BinaryClassificationEvaluator`

Tuning

Choose params with validation data

`ParamGridBuilder`, `CrossValidator`, `TrainValidationSplit`

Persistence

Save and load stages and models

`model.write().save(path)`, `PipelineModel.load(path)`

Green = used in the tutorial · pyspark.ml is the DataFrame API; RDD-based pyspark.mllib is in maintenance mode

Spark ML works on DataFrame columns

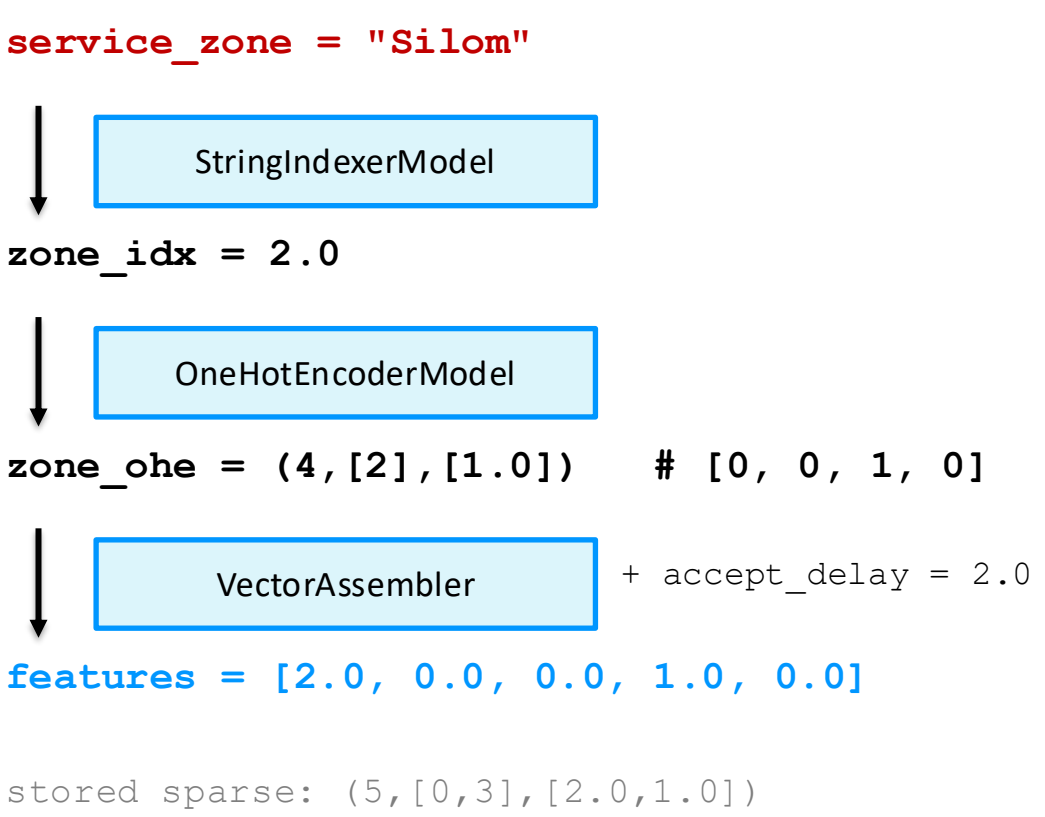
- Spark ML (pyspark.ml) is DataFrame-based: each step reads columns and adds new ones
- A model reads one vector column, features, and adds a prediction column

raw data			feature vector	label	prediction
service_zone	accept_delay_minutes	is_rainy	features	driver_arrival_wait_minutes	prediction
Ari	2.0	0.0	[2.0, 0.0]	6.0	6.93
Silom	2.0	1.0	[2.0, 1.0]	13.0	11.16

VectorAssembler adds features · the fitted LinearRegressionModel adds prediction

Feature vectors: dense and sparse

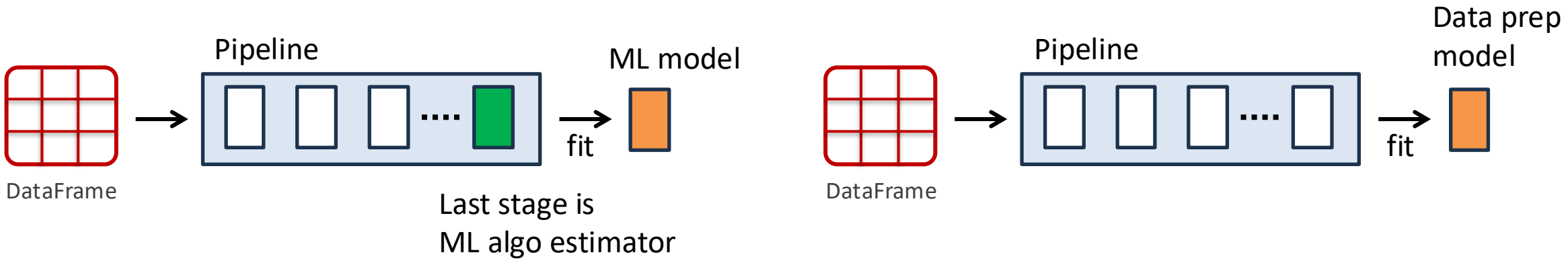
- A model reads one vector of numbers per row
- Dense: stores every value
  - `[2.0, 0.0, 0.0, 1.0, 0.0]`
- Sparse: size, positions, values
  - `(5, [0, 3], [2.0, 1.0])`
- One-hot vectors are mostly zeros, so Spark often stores them sparse
- Model 2 has 14 features: delay 1 + zone 4 + slot 2 + weekday 7



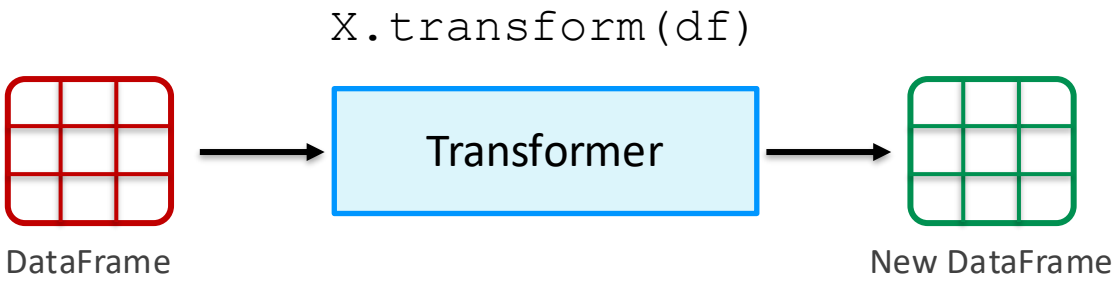
CHULA ENGINEERING
Foundation toward Innovation

Spark ML Concepts

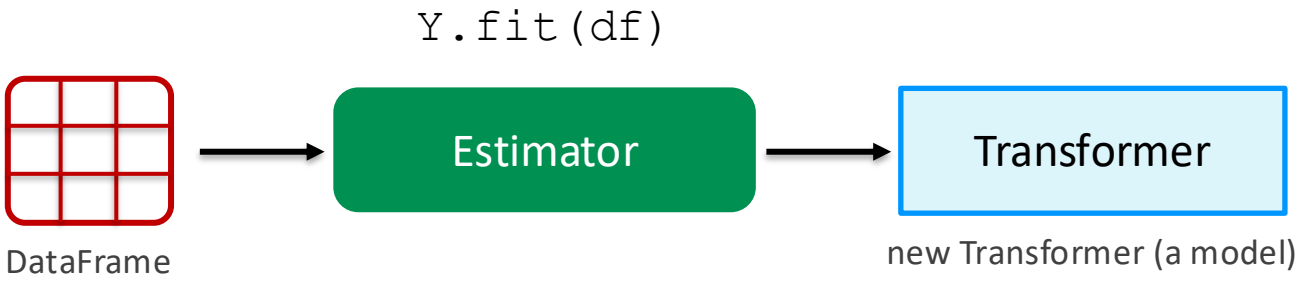
- Spark ML manipulates DataFrames with a transformer model
- Pipeline is a chain of operations with one or more stages
- Each pipeline stage can be either
  - (Spark ML) Transformer: transform dataframe to dataframe
  - Estimator: fit transformer from dataframe
- Pipeline is an estimator; use for creating an ML or data prep model



Transformer and Estimator



adds, changes or removes columns



learns parameters from the data, then returns a Transformer that applies them

In the tutorial

Estimator: fit()	Transformer: transform()
StringIndexer	StringIndexerModel
OneHotEncoder	OneHotEncoderModel
LinearRegression	LinearRegressionModel
Pipeline	PipelineModel
	VectorAssembler

Each Estimator on the left returns the Transformer on its right.

CHULA ENGINEERING
Foundation toward Innovation

Params: configure and inspect every stage

- Every stage has named params with defaults
  - featuresCol, labelCol, maxIter, regParam, …
- Set them when you create the stage
- Read them back with get…() or explainParam()
- A fitted model exposes what it learned
  - labels, intercept, coefficients
- fit() never changes the Estimator: it returns a new model

```
lr = LinearRegression(featuresCol="features",
                      labelCol=LABEL)
lr.getMaxIter()         # 100
lr.getRegParam()        # 0.0
print(lr.explainParam("regParam"))
# regParam: regularization parameter
#   (>= 0). (default: 0.0)

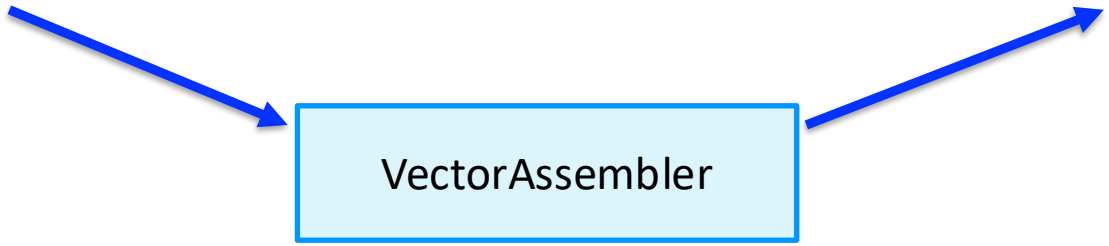
lr4 = m4.stages[-1]     # rain model
lr4.intercept           # 5.17
lr4.coefficients        # [0.88, 4.23]

m2.stages[0].labelsArray[0]
# ['Siam', 'Asok', 'Silom', 'Ari']
```

Transformer example: VectorAssembler

- VectorAssembler packs feature columns into one vector column

accept_delay_minutes	is_rainy	wait
2.0	0.0	6.0
2.0	1.0	13.0



accept_delay_minutes	is_rainy	wait	features
2.0	0.0	6.0	[2.0, 0.0]
2.0	1.0	13.0	[2.0, 1.0]

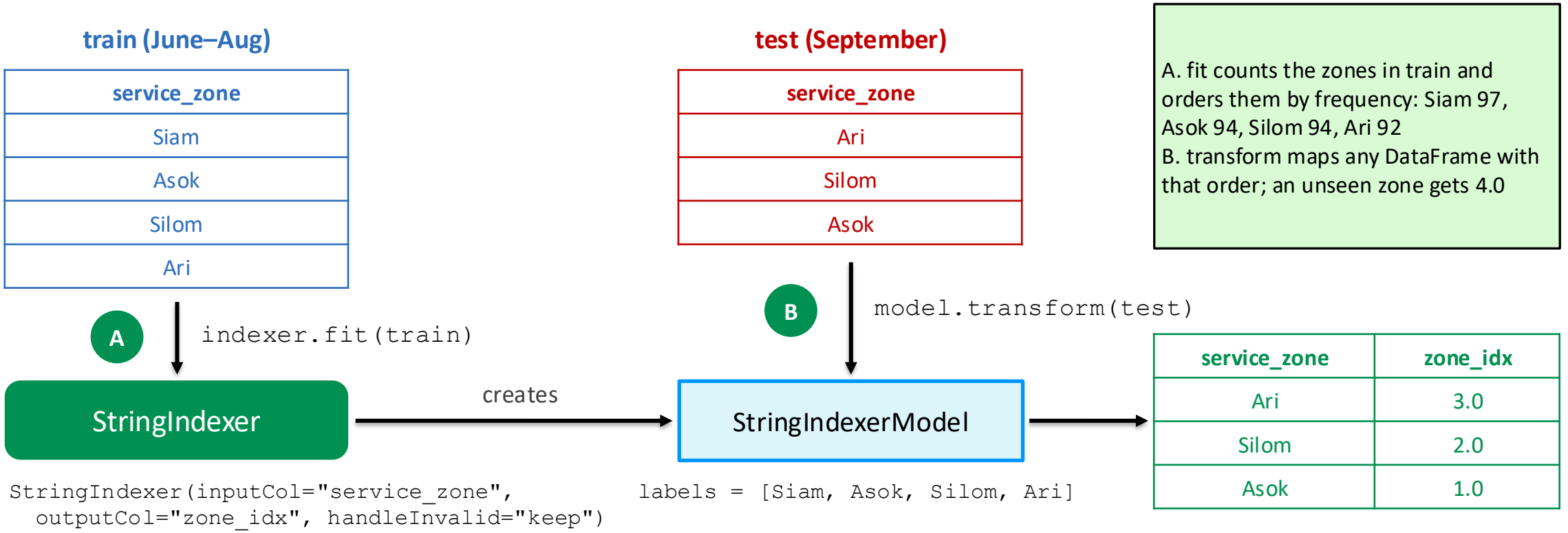
```
VectorAssembler(inputCols=["accept_delay_minutes", "is_rainy"], outputCol="features")
```

No fit(): there is nothing to learn, so it is a plain Transformer

CHULA ENGINEERING
Foundation toward Innovation

Estimator example: StringIndexer

- StringIndexer learns a category → index mapping from the training data

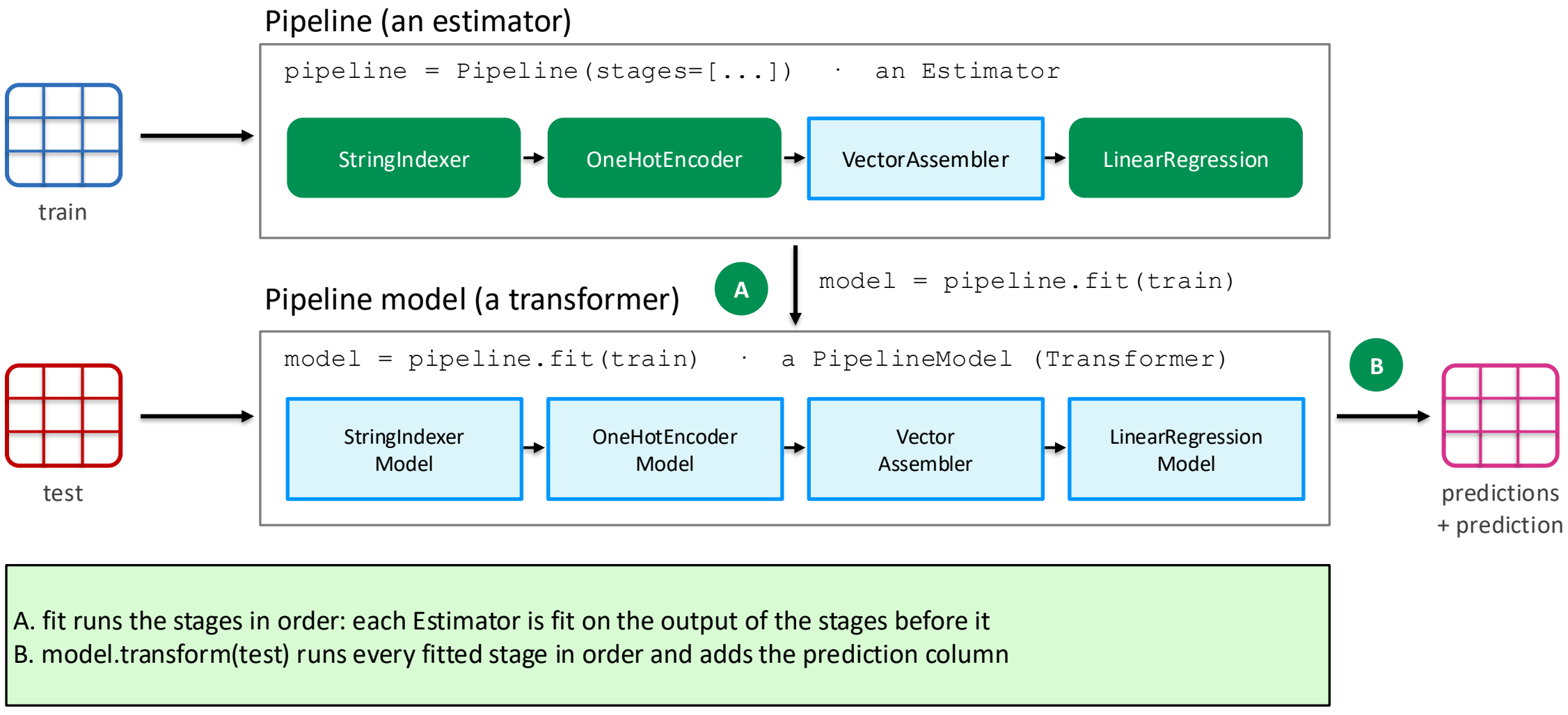


Common feature stages

Stage	Kind	What it does	In tutorial
`StringIndexer`	**Estimator**	category → index, most frequent first	yes
`OneHotEncoder`	**Estimator**	index → 0/1 vector (learns the number of categories)	yes
`VectorAssembler`	**Transformer**	columns → one features vector	yes
`StandardScaler`	**Estimator**	scale to unit variance (learns mean and std)	
`MinMaxScaler`	**Estimator**	scale to [0, 1] (learns min and max)	
`Imputer`	**Estimator**	fill missing values with the mean or median	
`Bucketizer`	**Transformer**	number → bucket, using fixed splits	
`SQLTransformer`	**Transformer**	derive columns with SELECT … FROM __THIS__	
`Tokenizer, HashingTF`	**Transformer**	text → words → term-frequency vector	

If a stage learns from the data, it is an Estimator: fit it on the training data only.

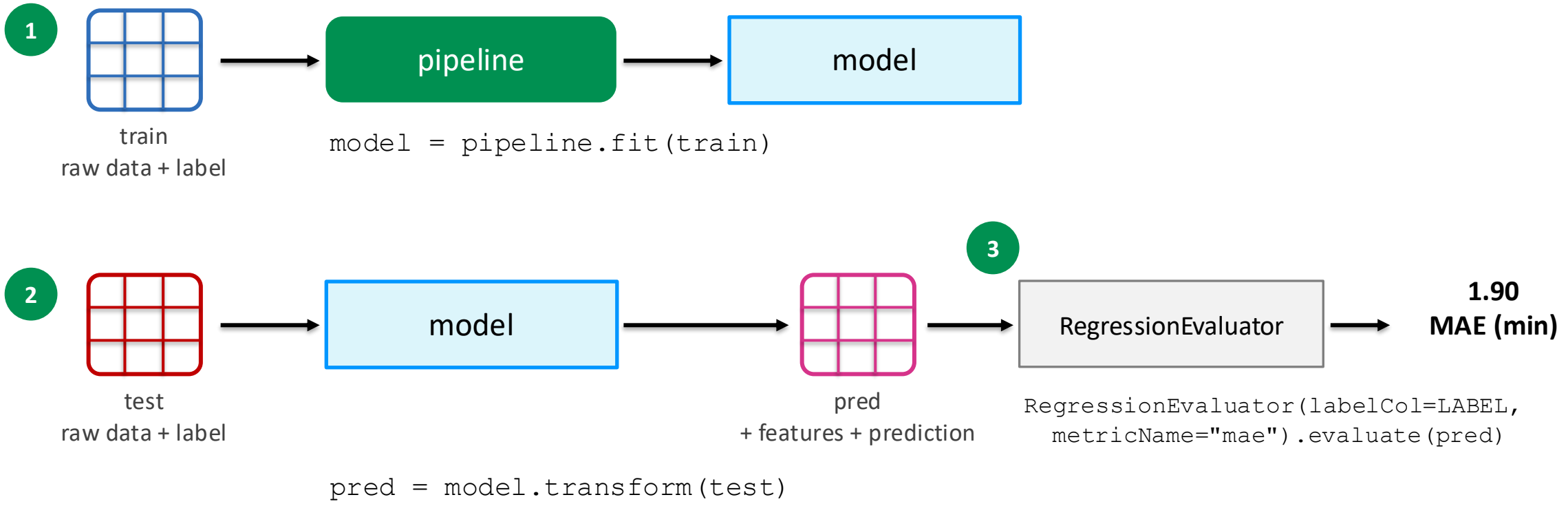
Pipeline: an Estimator made of stages



A. fit runs the stages in order: each Estimator is fit on the output of the stages before it
B. model.transform(test) runs every fitted stage in order and adds the prediction column

CHULA ENGINEERING
Foundation toward Innovation

Operation sequence: fit, transform, evaluate



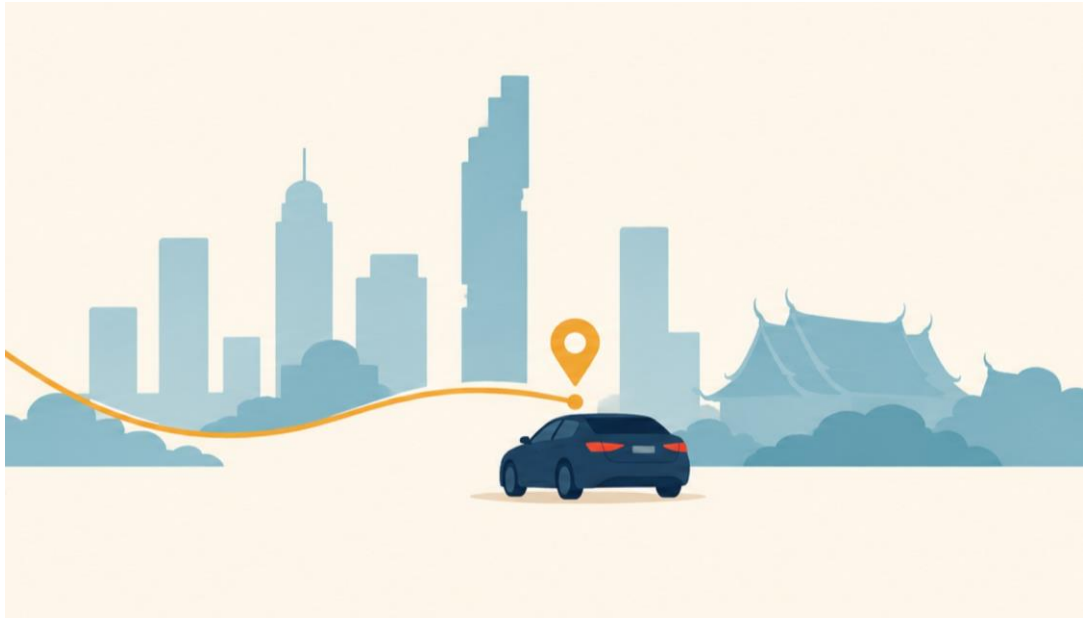
4 `lr = model.stages[-1];  lr.intercept, lr.coefficients   # 5.68, [1.15, …]`

Fit only on train. Look at the test set once, at the end.

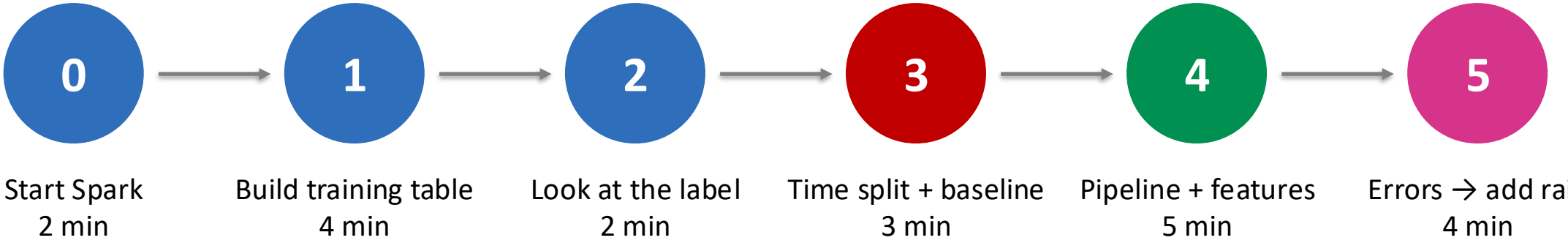
CHULA ENGINEERING
Foundation toward Innovation

Part 3

The tutorial, step by step



Tutorial: 6 steps in 20 minutes




```
cd case/ride-hailing/ml           # reads ../phases/*/bronze/*.csv
pip install pyspark "pandas<3"    # needs Java 17 or newer
jupyter lab tutorial_spark_ml.ipynb    # runs top to bottom in about a minute
```

Spark and data evaluation Spark ML pipeline error analysis

Step 1 · One training row per completed trip

1 Completed trips

Trip CSVs, status = completed; parse timestamps

2 Join arrivals

Inner join: trips without an arrival event have no label

3 Join rainfall

Left join by date and slot: June–August get NULL

Column	Role
zone, slot, weekday	categorical features, known at acceptance
accept_delay_minutes	numeric feature: accepted_at − requested_at
is_rainy	numeric feature: 1.0 if rain_mm ≥ 1.0 (September only)
driver_arrival_wait_minutes	label: requested_at → arrived_at_pickup

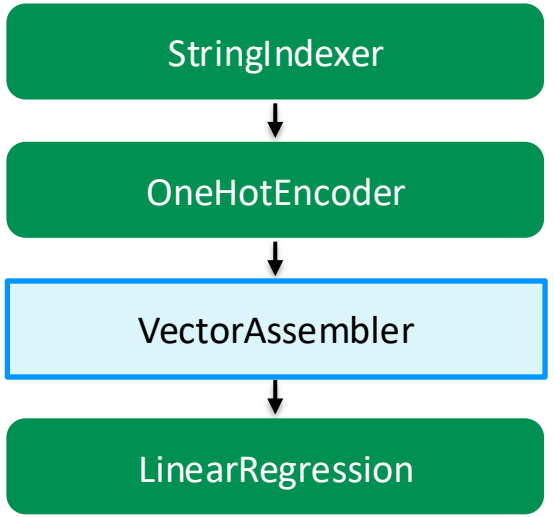
501

labelled trips; cancelled trips and April–May have no label

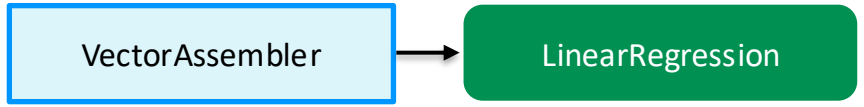
Step 4 · One helper builds every pipeline

```
def fit_and_score(name, categorical, numeric,
                  train_df=train, test_df=test):
    stages = []
    if categorical:                          # Estimators
        stages += [
            StringIndexer(inputCols=categorical,
                outputCols=[c + "_idx" for c in categorical],
                handleInvalid="keep"),
            OneHotEncoder(
                inputCols=[c + "_idx" for c in categorical],
                outputCols=[c + "_ohe" for c in categorical])]
    stages += [
        VectorAssembler(                     # Transformer
            inputCols=numeric + [c + "_ohe" for c in categorical],
            outputCol="features"),
        LinearRegression(featuresCol="features", labelCol=LABEL)]
    model = Pipeline(stages=stages).fit(train_df)
    score(name, model.transform(test_df))
    return model
```

Model 2: categorical + numeric

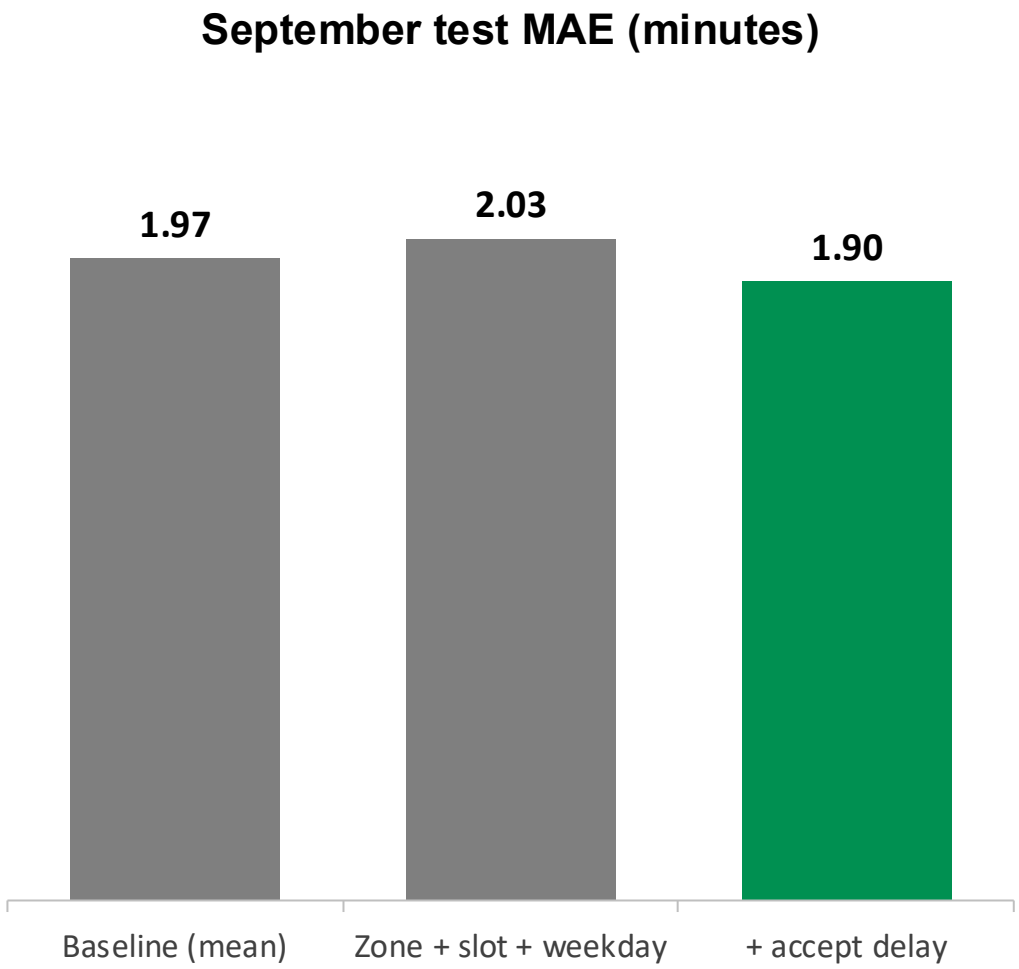


Rain model: numeric only



one call = one Pipeline + one score

Step 4 · The obvious features carry no signal



Baseline: predict the training mean → MAE 1.97

```
prediction = F.lit(train_mean)
```

Model 1: zone, slot, weekday → MAE 2.03

```
fit_and_score("zone + slot + weekday",
  ["service_zone", "time_slot", "weekday"], [])
```

Model 2: + acceptance delay → MAE 1.90

```
fit_and_score("+ accept delay", [...],
  ["accept_delay_minutes"])
```

Inside Model 2

```
lr = m2.stages[-1]
lr.intercept, lr.coefficients[0]   # 5.68, 1.15
```

Step 5 · The errors point to rain

+3.0

minutes: on rainy slots, drivers arrive later than Model 2 predicts

1.24

minutes MAE with rain (1.79 without), September only

Model 2 residuals on September, by weather

weather	trips	avg residual (min)
dry	92	−1.04
rainy	32	+3.04

September only: train 1–20, test 21–30

model	MAE	R^2
baseline	1.98	0.00
accept delay	1.79	0.15
accept delay + rain	1.24	0.64

```
wait ≈ 5.17 + 0.88 × accept delay + 4.23 if rainy
```

Part 4

Beyond the tutorial



Other algorithms, same pattern

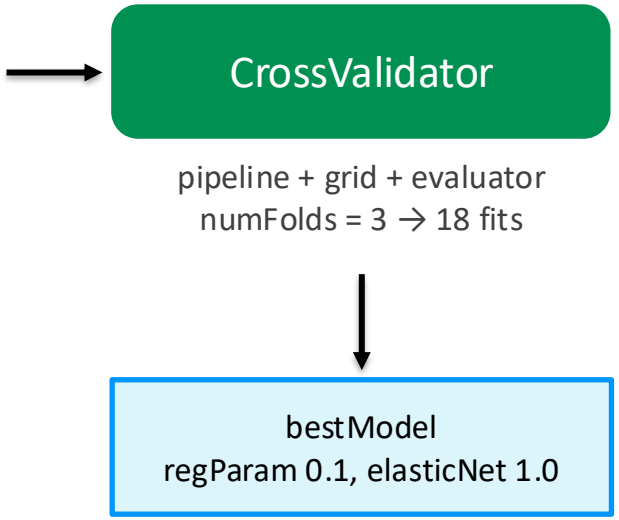
Task	Estimators	Evaluator (metric)	MetroRide question
Regression	`LinearRegression`, `RandomForestRegressor`, `GBTRegressor`	`RegressionEvaluator (mae, rmse, r2)`	How many minutes until the driver arrives? (tutorial)
Binary classification	`LogisticRegression`, `RandomForestClassifier`, `GBTClassifier`	`BinaryClassificationEvaluator (areaUnderROC)`	Will the driver arrive within 8 minutes? Will the trip be cancelled?
Multiclass	`LogisticRegression`, `RandomForestClassifier`, `NaiveBayes`	`MulticlassClassificationEvaluator (f1, accuracy)`	Will the wait be short, medium or long?
Clustering	`KMeans, BisectingKMeans, GaussianMixture`	`ClusteringEvaluator (silhouette)`	Which pickup GPS points form hotspots?
Recommendation	`ALS`	`RegressionEvaluator (rmse)`	Which destinations suit a passenger?

Swap the last stage and the evaluator; the feature stages stay the same.

Tuning: ParamGridBuilder and CrossValidator

6 param combinations

regParam	elasticNet
0.0	0.0
0.0	1.0
0.1	0.0
0.1	1.0
1.0	0.0
1.0	1.0




```
grid = (ParamGridBuilder()
    .addGrid(lr.regParam, [0.0, 0.1, 1.0])
    .addGrid(lr.elasticNetParam, [0.0, 1.0])
    .build())                          # 6 maps
cv = CrossValidator(estimator=pipeline,
    estimatorParamMaps=grid,
    evaluator=RegressionEvaluator(
        labelCol=LABEL, metricName="mae"),
    numFolds=3, seed=42)
best = cv.fit(train).bestModel    # 18 fits
```

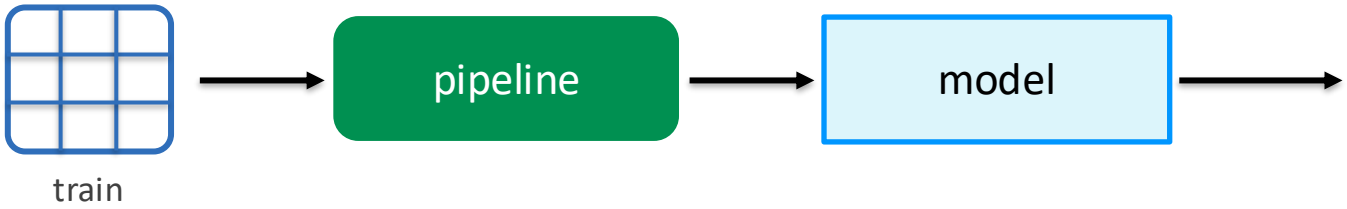
Test MAE 1.86 (was 1.90). Lasso (elasticNet 1.0) set 10 of the 14 weights to zero: zone, slot and weekday carry no signal.

Careful with time

CrossValidator and TrainValidationSplit split rows at random. For MetroRide, validate on a later month instead: train June–July, validate August, test September.

Saving and loading a fitted pipeline

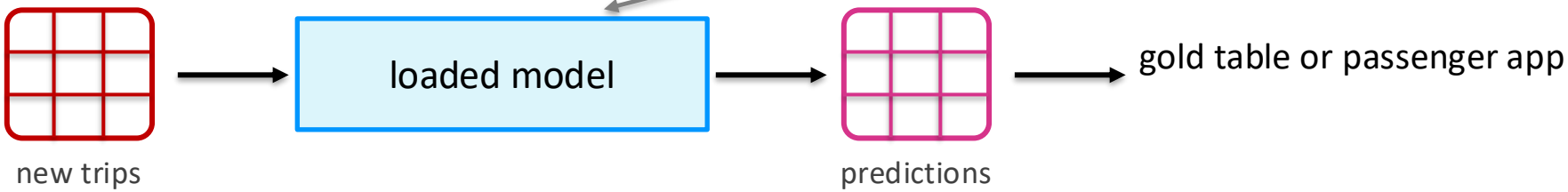
Training notebook




```
models/arrival_wait/
  metadata/
  stages/0_StringIndexer_…/
  stages/1_OneHotEncoder_…/
  stages/2_VectorAssembler_…/
  stages/3_LinearRegression_…/
```

```
model.write().overwrite().save("models/arrival_wait")
```

Scoring job (nightly batch or streaming)



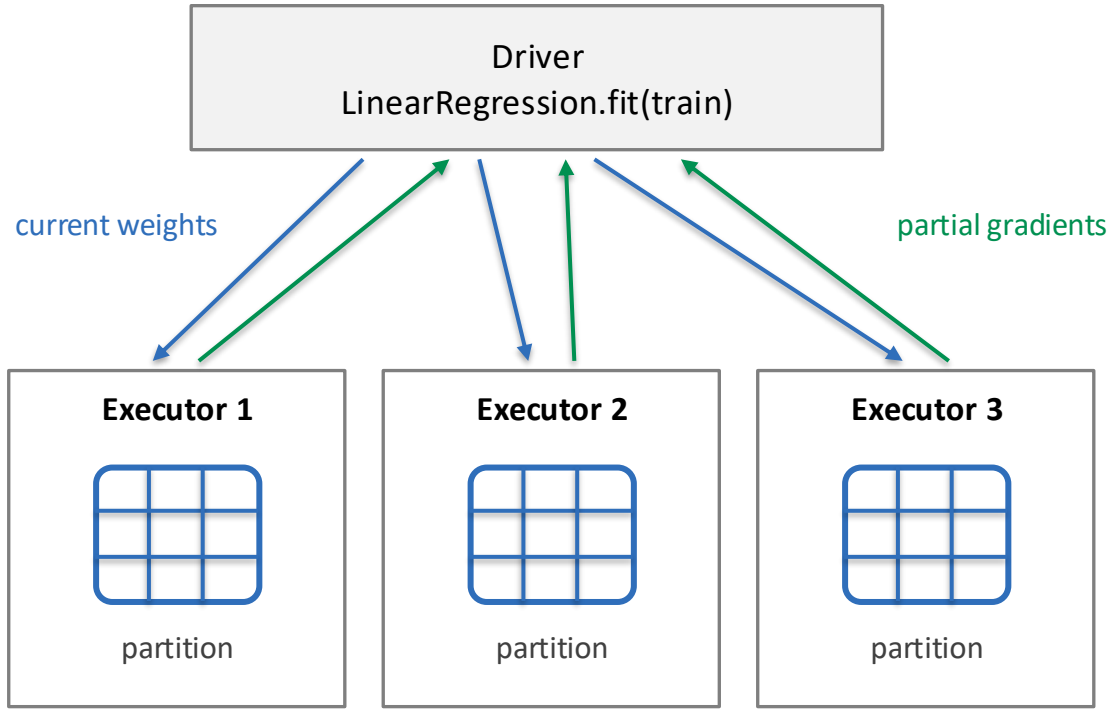

```
model = PipelineModel.load("models/arrival_wait");   model.transform(new_trips)
```

The loaded model gives the same September MAE, 1.90: every fitted stage is saved.

CHULA ENGINEERING
Foundation toward Innovation

Spark ML at scale

- fit() and transform() run as Spark jobs on partitioned data
- Feature stages work row by row, so they scale out easily
- Training is iterative: many passes over the data
  - cache() the training DataFrame; maxIter limits the passes
- Tuning multiplies the work: 6 × 3 = 18 fits
- MetroRide has 501 rows: pandas + scikit-learn would be simpler
  - Spark ML pays off when data no longer fits one machine



repeat until converged or maxIter is reached

CHULA ENGINEERING
Foundation toward Innovation

Where each concept appears in the tutorial

Concept	Spark ML API	Tutorial step
Label, features, leakage	`DataFrame columns`	1 · Build the training table
Time split and baseline	`filter(month), F.lit(train_mean)`	3 · Split by time
Metrics	`RegressionEvaluator(metricName=...)`	3 · Score a baseline
Transformer and Estimator	`VectorAssembler; StringIndexer, OneHotEncoder`	4 · fit_and_score()
Feature vectors	`features, sparse one-hot columns`	4 · fit_and_score()
Pipeline and PipelineModel	`Pipeline(stages).fit(), model.transform()`	4–5 · fit_and_score()
Params and fitted values	`stages[-1].intercept, .coefficients`	4–5 · Read the weights
Error analysis	`groupBy(is_rainy) on residuals`	5 · Add rain

Wrap-up

Key ideas

- Frame the problem first: label, features known at prediction time, time split, baseline
- Spark ML stages are Transformers or Estimators; a Pipeline chains them
- Fit on train only, transform anything, evaluate once on test
- Read errors and fitted params to find the next feature
- Tune on later data; save the PipelineModel to reuse it

Try it yourself

1. Leakage: add the passenger pickup wait as a feature. Why does MAE drop to 0.00?
2. Swap LinearRegression for RandomForestRegressor
3. Classify “arrives within 8 minutes” with LogisticRegression
4. Tune regParam on August, then test on September
5. Save the best PipelineModel and load it in a new notebook